Direct Dispatch Optimizer

Product Requirements Document

A supply chain optimization tool that finds hidden savings in how products move from factory to customer.

The Problem

Most large distributors move products in two hops: Factory → Distribution Center → Customer. The Distribution Center (DC) exists to consolidate and re-sort inventory - but for certain large, predictable orders, that extra hop adds unnecessary transport cost, extra handling, and extra time, without adding any real value.

The alternative - Direct Dispatch (DD): shipping straight from the factory to the customer, skipping the DC entirely - can save money and reduce carbon emissions, but only when a specific set of conditions line up: the order has to be large enough to fill a truck, the factory has to have enough stock on hand, and the economics have to actually work out in the company's favor.

Deciding which orders qualify, how much stock a factory should hold back for this purpose, and whether it's worth it is a genuinely hard operations research problem - not something that can be solved with a simple spreadsheet rule.

The Solution

This project is a working proof-of-concept web application that:

- Evaluates every sales order against a chain of real-world business rules (minimum truck load, product availability, warehouse space) to determine if it's a Direct Dispatch candidate.
- Calculates the actual dollar savings (or cost) of shipping that order directly, based on real distance and transport cost data.
- Optimizes how much inventory each factory should hold back for direct shipping - too little wastes the opportunity, too much ties up warehouse space that has a real cost.
- Compares 9 different business scenarios side by side (different order-consolidation windows, with and without warehouse capacity limits) so decision-makers can see the trade-offs before committing to a policy.

How It Works, In Plain Terms

Think of it like a smart triage system for every order that comes in:

- Step 1 - Is the order big enough? A truck only makes financial sense if it's sufficiently full. Small orders get combined with other nearby orders (within a 1, 3, or 7-day window) to see if together they clear the bar.
- Step 2 - Does the factory have the stock? Even a large, eligible order can't ship direct if the factory hasn't set aside enough of that product.
- Step 3 - Is it actually worth it? Every direct shipment is checked against the traditional route's cost. If shipping direct would cost more than the standard route, it doesn't happen - the tool only recommends Direct Dispatch when it genuinely saves money.
- Step 4 - How much should we stock ahead of time? This is the hardest part: for every product at every factory, the system searches for the ideal balance - enough inventory retained to catch future direct-ship opportunities, without over-committing warehouse space that has its own cost.

Key Features

- 9-scenario comparison matrix - instantly compare outcomes across different consolidation windows and warehouse capacity assumptions
- Order-level explainability - for any order, see exactly why it did or didn't qualify for Direct Dispatch (which rule it passed or failed)
- Per-product allocation policy - a clear view of what percentage of each product's stock is recommended to stay at the factory vs. flow through the normal DC route
- Site-level performance breakdown - cost savings, fill rate, and warehouse utilization by factory
- CO2 impact tracking - every scenario also reports estimated carbon savings from reduced transport distance

Business Impact (from POC Testing)

Using a synthetic dataset modeled on realistic order volumes, product mix, and shipping costs, the optimizer identified:

- **Up to ~$6,31,720 in net annual savings in the best-performing scenario**
- **Up to ~241,000 kg of CO2 saved annually through reduced transport distance**
- Clear, quantified trade-offs between more aggressive consolidation (higher savings, more planning complexity) and simpler daily-only policies

(All figures are from a synthetic test dataset built to validate the model's logic - not a live company's actual data.)

Validation & Quality Assurance

Because this is an optimization engine making real financial recommendations, correctness mattered more than a typical prototype. The project includes:

- A User Acceptance Testing (UAT) plan with concrete, hand-verifiable test cases - including real worked examples showing exactly why specific orders passed or failed each business rule
- Independent verification of the optimizer's core decisions (not just trusting the dashboard) - recomputing the "ideal" stock allocation for individual products from raw data and checking it against what the app actually produced
- Iterative bug-hunting that caught and fixed real issues along the way, including a case where the app was technically completing shipments but losing money on them

Key Challenges & How We Solved Them

Building this wasn't a straight line. Several rounds of UAT surfaced real, consequential problems - the kind that don't show up until you stop trusting summary numbers and start checking the underlying logic by hand. These are the five biggest ones.

1. Results that contradicted common sense

Early testing showed something that shouldn't have been possible: giving the system more time to combine small orders into full truckloads was reducing projected savings instead of increasing them. We traced the order-grouping logic line by line and found it was combining orders into fixed calendar windows (e.g. rigid Monday-to-Sunday buckets) instead of a rolling window anchored to each individual order's date - a subtle bug that quietly split up orders that should have been grouped together. Fixing the window logic restored the expected pattern: more flexibility to combine orders should never make the outcome worse, and after the fix, it didn't.

2. The test data was working against the model, not for it

Even once the optimizer's logic was solid, the numbers still came out negative almost everywhere. Rather than assume the code was still broken, we went back to the underlying data. It turned out customer and factory locations had been placed randomly with no geographic logic - meaning "skip the distribution center" often meant a longer delivery trip, not a shorter one. We rebuilt the test dataset so customer locations, factory locations, and shipping costs all followed realistic real-world patterns, and the economics immediately made sense.

3. Learning not to trust a "fixed and verified" status report

Several times during development, the AI coding tool reported that a bug had been fixed and even produced specific, convincing-looking test numbers to prove it - but the live, running application showed completely different, unchanged results. We stopped accepting these summaries at face value. Instead, we requested the literal, unmodified source code every time, read through the logic directly, and independently recalculated the app's own numbers in a separate script using the same raw data, to check whether the two actually agreed. This became the standard process for the rest of the project: verify with evidence, not with a status update.

4. A capacity-limited search that was stuck from the very first step

Under one specific setting - a limited-warehouse-space scenario - the system consistently produced almost no usable results, regardless of what else changed. Tracing the search process revealed the cause: every product's recommended stock level started at a high value at the same time, instantly overloading the shared warehouse space before the search had a chance to explore any real

trade-offs. Once space was gone, the search had no path to recover. The fix was to start conservatively and ration the limited space fairly, based on which products were actually worth prioritizing.

5. The most serious find: shipments that lost money and no one would have noticed

The most important issue surfaced almost by accident. Total savings for one scenario came out slightly negative, which was strange given everything else looked healthy. Rather than accept the aggregate number, we pulled the individual order-level detail - and found the real cause: the system was approving a Direct Dispatch shipment any time it was large enough and stock was available, without ever checking whether shipping direct was actually cheaper than the normal route. Some approved shipments were losing over $40 per pallet. We added an explicit profitability check before any order is recommended for Direct Dispatch, and corrected the underlying scoring logic so that correctly declining a bad deal counts as a win, not a penalty. This is the kind of bug that a dashboard showing only totals would never reveal - it only surfaces when you check individual decisions against their real-world economics.

The common thread: aggregate numbers and confident status reports can both look fine while hiding a real problem underneath. The validation approach that actually worked was always the same - go to the individual order level, recompute the answer independently, and only trust what can be verified against raw data.

Tech Stack

Layer	Technology
Frontend	React, TypeScript, Tailwind CSS
Backend	Node.js
Optimization approach	Custom simulation-and-search algorithm (coordinate ascent) - the engine simulates a full year of daily operations under different stock-allocation policies and searches for the policy that maximizes savings
Built with	Google AI Studio (Build mode), with the underlying business logic, data model, and validation methodology designed and specified independently

Skills Demonstrated

- Translating a real, ambiguous business problem into a precise, testable technical specification
- Supply chain / operations research concepts: constrained optimization, cost modeling, capacity planning
- Synthetic data generation for realistic testing and demonstration
- Rigorous validation methodology - building independent checks rather than trusting a system's own self-reported results
- Iterative, evidence-based debugging of a complex, multi-component application
- Clear technical communication for both technical (engineering) and non-technical (business) audiences

Project Links

- Live demo: Direct Dispatch Optimizer
- GitHub repository: Direct Dispatch Optimizer GitHub
- UAT Plan and sample dataset included in the repository's docs/ folder

This is a proof-of-concept project built to explore and demonstrate supply chain optimization techniques. All data is synthetic.